

BCC1313 ALGORITHMS AND COMPLEXITY

Queue Management

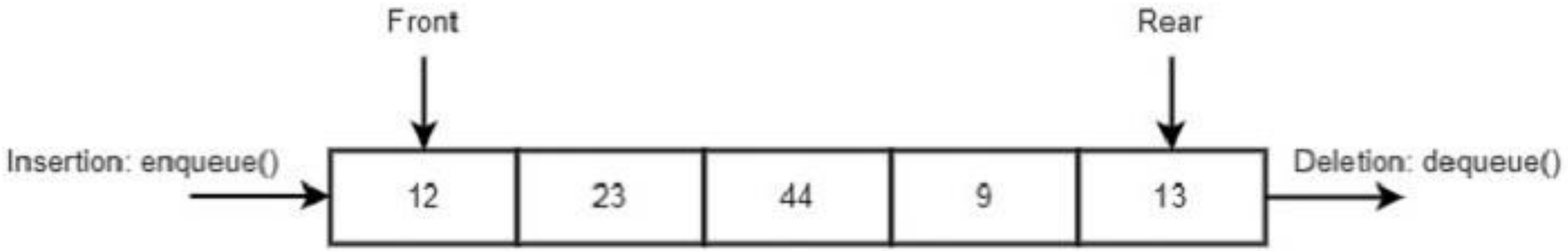
INTRODUCTION

- A **queue** is a linear data structure that follows the **First In, First Out (FIFO),** where the first item added is the first one to be removed.

- We encounter various types of queues in our daily lives, such as waiting in line at a bank or queueing for customer service.

QUEUES

- A **queue** stores data similarly to an **array**, with the following three key constraints:
    1. Data can only be **inserted at the rear of the queue.**
    2. Data can only be **read from the front of the queue.**
    3. Data can only be **removed from the end of a queue.**



QUEUE OPERATIONS

- ADT queue operations:
  - **Create an empty queue**
  - **Destroy a queue**
  - **Determine whether a queue is empty**
  - **Add a new item to the queue (enqueue)**
  - **Remove the item that was added earliest (dequeue)**
  - **Retrieve the item that was added earliest.**

QUEUE OPERATIONS

- Operation contract for ADT queue:
  - Create an empty queue and delete a queue

```
createQueue()
destroyQueue()
```

  - Determine whether this queue is empty

```
isEmpty():boolean (query)
```

QUEUE OPERATIONS

- Operation contract for ADT queue:
  - Removes the front of the queue for the item that was added earliest

```
deQueue()  throw QueueException
//throw QueueuException if deletion is not successfull
```

  - Retrieve into queueFront and then remove the front of the queue

```
deQueue (out queueFront: QueueItemType)
throw QueueException
```

QUEUE OPERATIONS

- Operation contract for ADT queue:
  - Retrieve into queueFront the front of the queue

```
getFront (out queueFront: QueueItemType)
throw QueueException
```

  - Insert item into the front of the queue

```
enQueue() throw QueueException
```

QUEUE OPERATIONS

- Example of queue operations with integers:

Operation	Queue after operation (front →)
`aQueue.createQueue()`	
`aQueue.enqueue(5)`	5
`aQueue.enqueue(2)`	5 2
`aQueue.enqueue(7)`	5 2 7
`aQueue.getFront(queueFront)`	5 2 7 (queueFront is 5)
`aQueue.dequeue(queueFront)`	2 7 (queueFront is 5)
`aQueue.dequeue(queueFront)`	7 (queueFront is 2)

QUEUE APPLICATION

- **Example 1: Reading a String of Character**
  - A queue can retain characters in the order in which they are typed:

```
aQueue.createQueue()
while (not end of line)
{   Read a new character ch
    aQueue.enqueue(ch)
} // end while
```

  - Once the characters are in a queue, the system can process them as necessary

QUEUE APPLICATION

- For example, convert typed digits into the decimal value:

  ‘247’ → 247

  2 + (0 * 10) = 2

  4 + (2 * 10) = 24

  7 + (24 * 10) = 247

```
// get first digit, ignoring any
// leading blanks
do
{   aQueue.dequeue(ch);
}   while (ch is blank)

// compute n from digits in queue
n=0
done = false
do {
      n = 10*n + integer that ch
         represents
      if (!aQueue.isEmpty())
         aQueue.dequeue(ch)
      else
         done = true
} while (!done and ch is a digit)
```

QUEUE APPLICATION

- **Example 2: Recognizing Palindromes**
  - A palindrome is a string of characters that reads the same from left to right as its does from right to left.

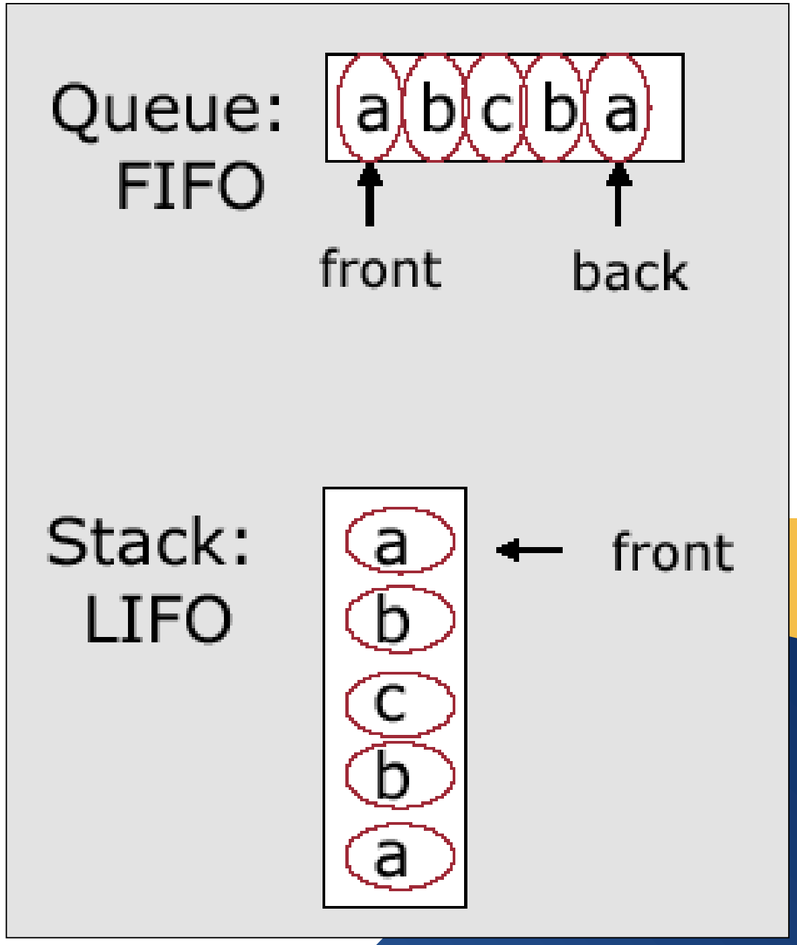
    abcba (yes) **abcbd (no)**
  - To recognize a palindrome, use a queue in conjunction with a stack
    - A **queue** preserve the order of occurrences
    - A **stack** preserve reverses the order of occurrences

QUEUE APPLICATION

- A recognition algorithm for palindrome:

  String : **a b c b a**

  - As traverse the character string from left to right, insert each character into both queue and stack.
  - Compare the characters at the front of the queue and the top of the stack.



QUEUE APPLICATION

- The algorithm:

```
isPal(in str:string):boolean

  aQueue.createQueue()
  aStack.createStack()

  length = length of str
  for(i = 1 through length)
  {
     nextChar = ith character of str
     aQueue.enqueue(nextChar)
     aStack.push(nextChar)
  }
```

QUEUE APPLICATION

- The algorithm:

```
charactersAreEqual = true
while(aQueue is not empty and charactersAreEqual)
{
   aQueue.getFront(queueFront)
   aStack.getTop(stackTop)

   if (queueFront equals stackTop)
   {   aQueue.dequeue()
       aStack.pop()
   }
   else
       charactersAreEqual = false
}

return charactersAreEqual
```

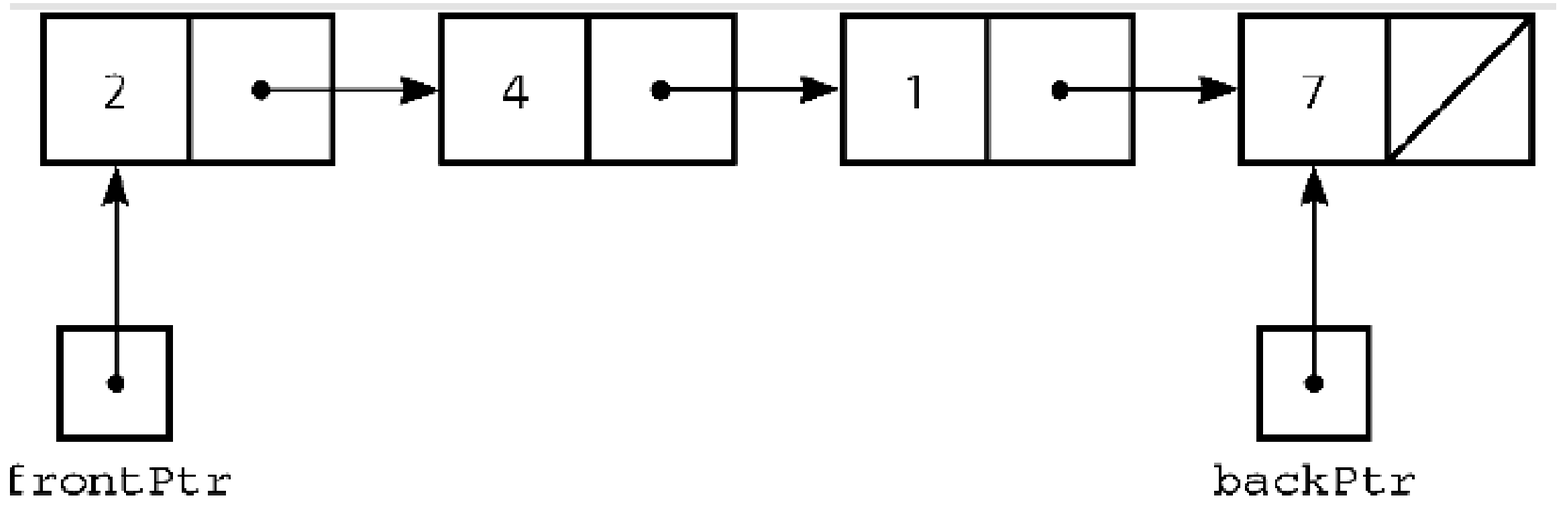
IMPLEMENTATION

- Like stacks, queues can have an **array-based implementation** or a **pointer-based implementation**.
- All of the implementations can use the following definition of QueueException:

```
class QueueException : public logic_error
{
Public:
        QueueException(const string & message = “”)
                : logic_error(message.c_str()) {}//end constructor
}; // end QueueException
```

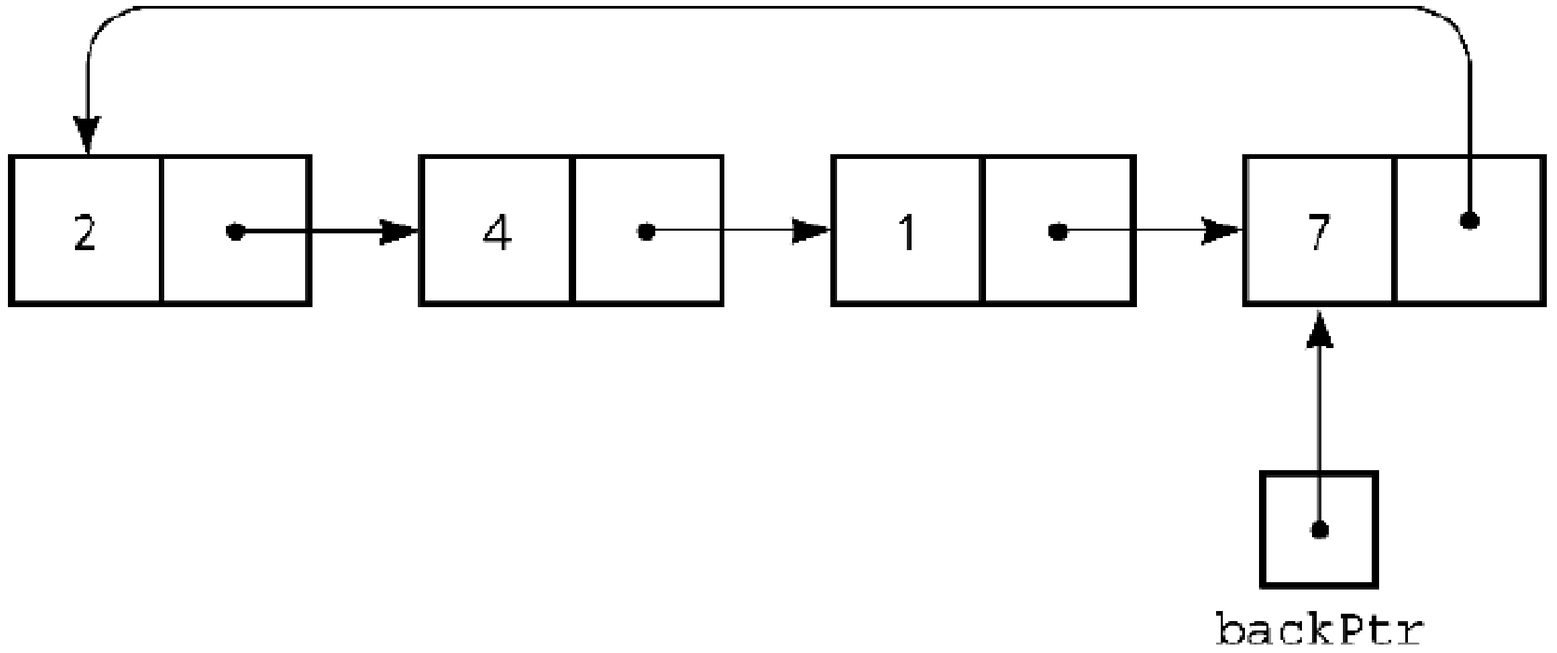
IMPLEMENTATION

- **A Pointer-based Implementation**
  - Possible implementations of a pointer-based queue have 2 external references: **front** and **back**.



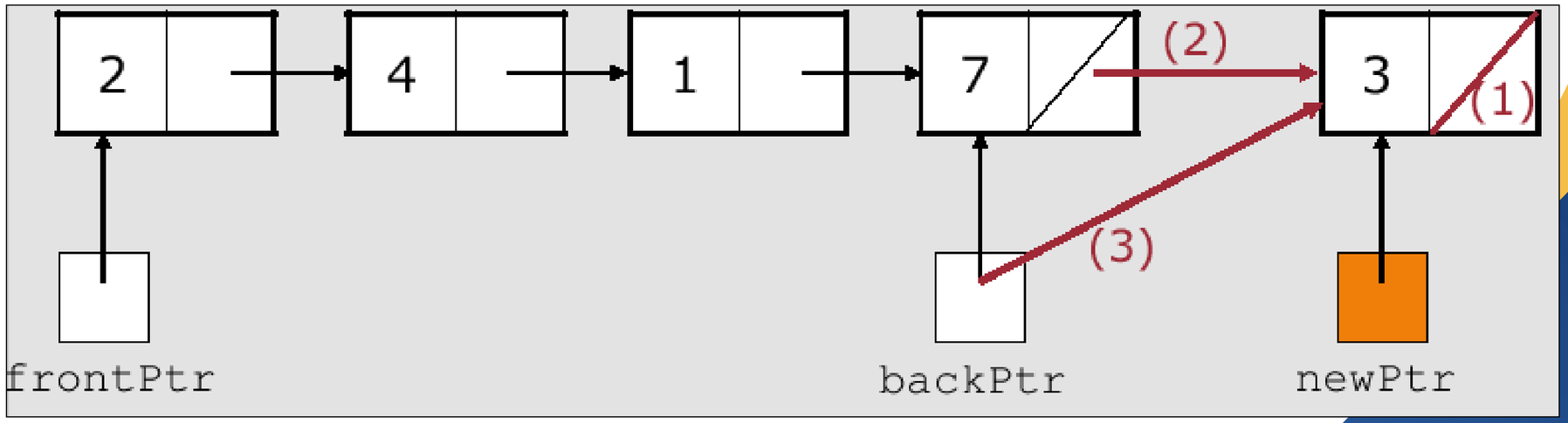
IMPLEMENTATION

- **A Pointer-based Implementation**
  - A circular linked list with one external reference: **back**



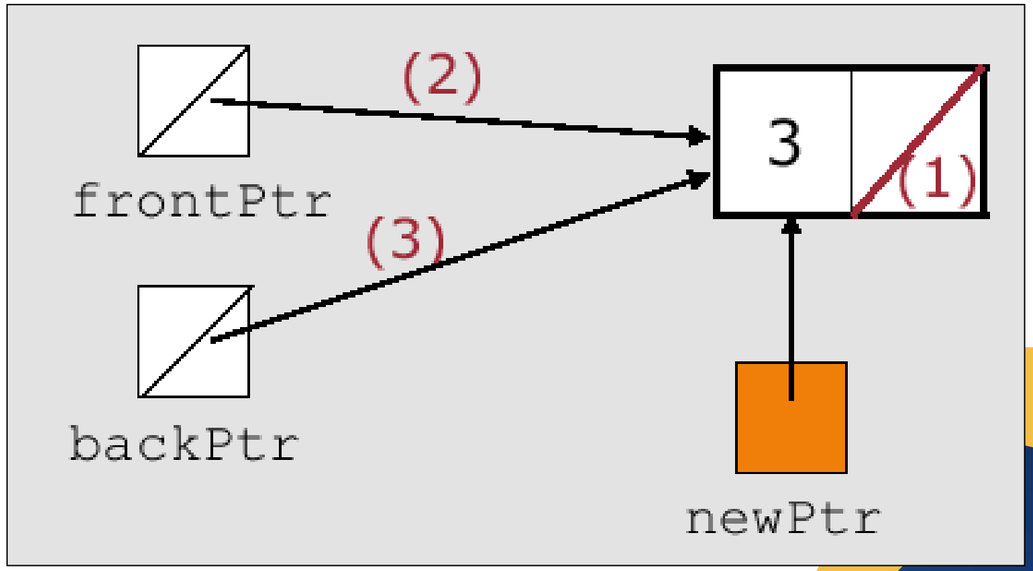
IMPLEMENTATION

- Insert a new node, to which newPtr points, at the back of a queue.
  - **Case 1: non-empty queue**
    1. **newPtr -> next = NULL;**
    2. **backPtr -> next = newPtr;**
    3. **backPtr = newPtr;**



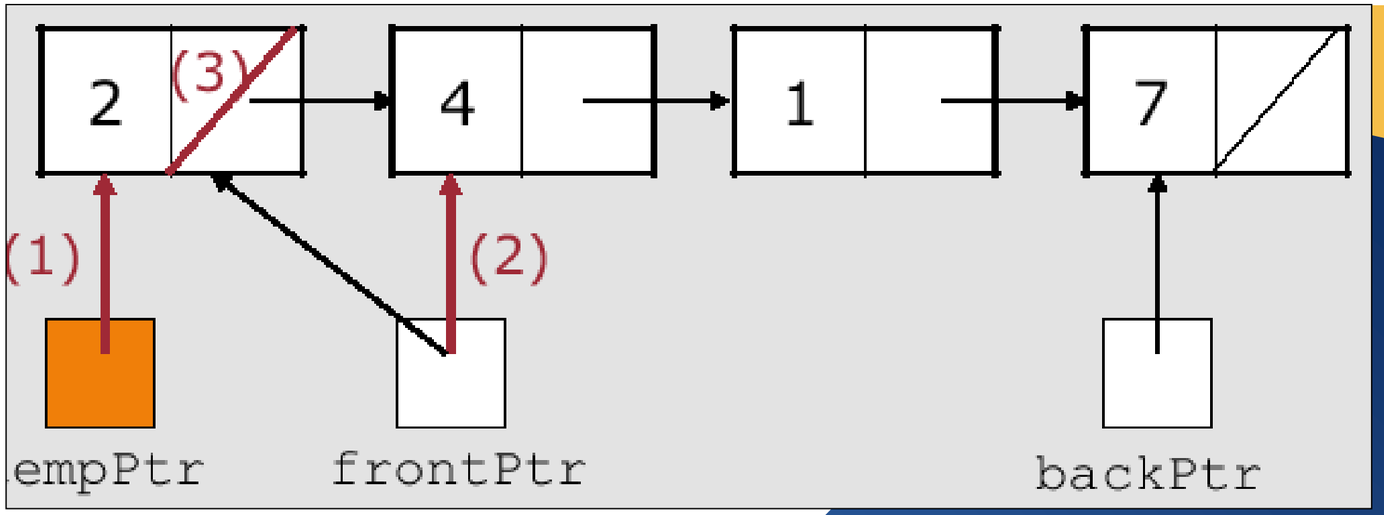
IMPLEMENTATION

- Insert a new node, to which newPtr points, at the back of a queue.
  - **Case 2: empty queue**
    1. **newPtr -> next = NULL;**
    2. **frontPtr = newPtr;**
    3. **backPtr = newPtr;**



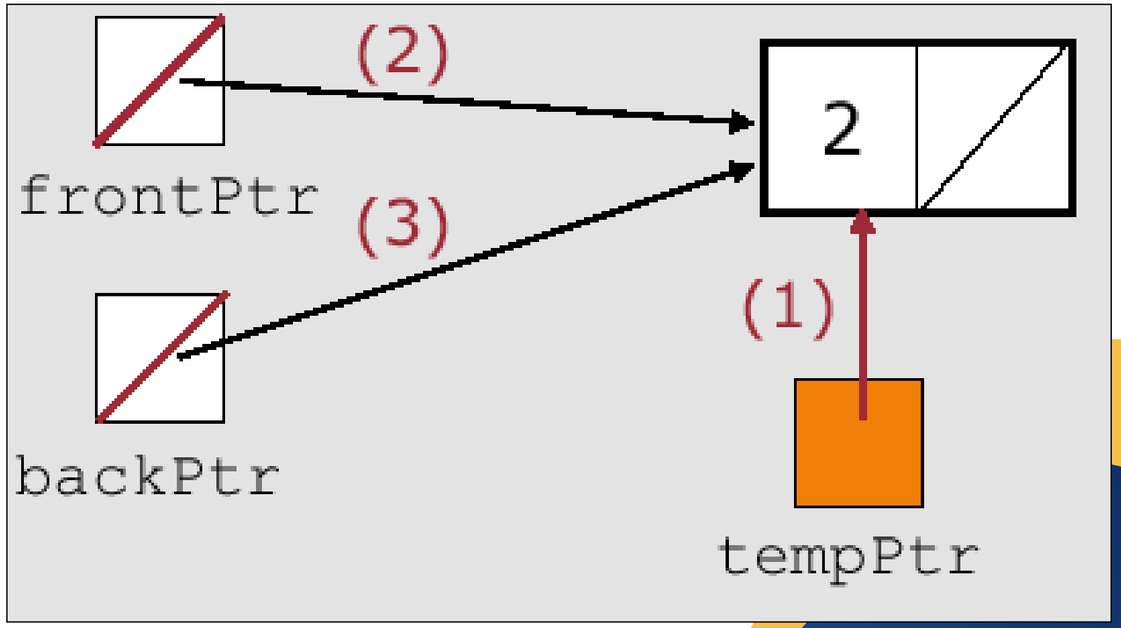
IMPLEMENTATION

- Remove the front item of a queue.
  - **Case 1: a queue contains more than one item:**
    1. **tempPtr = frontPtr;**
    2. **frontPtr = frontPtr -> next;**
    3. **tempPtr -> next = NULL;**
    4. **delete tempPtrl**



IMPLEMENTATION

- Remove the front item of a queue.
  - **Case 2: a queue of one item:**
    1. **tempPtr = frontPtr;**
    2. **frontPtr = NULL;**
    3. **backPtr = NULL;**
    4. **delete tempPtr;**



IMPLEMENTATION

- A pointer-based implementation of ADT queue

```
/** @file QueueP.cpp */
#include <cstddef>     // for NULL
#include <cassert>     // for assert
#include <new>         // for bad_alloc
#include "QueueP.h"    // header file
using namespace std;
Queue::Queue() : backPtr(NULL), frontPtr(NULL)
{} // end default constructor

Queue::Queue(const Queue&Q)
{ // implementation left as an homework!!
} // end copy constructor

Queue::~Queue()
{  while(!isEmpty())
     dequeue();
   assert( (backPtr == NULL) && (frontPtr == NULL) );
} // end destructor
```

```
bool Queue::isEmpty() const
{
  return backPtr == NULL;
} // end isEmpty

void Queue::getFront(QueueItemType& queueFront) const
                                    throw(QueueException)
{
  if(isEmpty())
    throw QueueException(
        "QueueException: empty queue, cannot getFront");
  else
    // queue is not empty; retrieve front
    queueFront = frontPtr->item;
} // end getFront
```

```
void Queue::enqueue(const QueueItemType& newItem) throw
(QueueException)
{
  try
  {  // create a new node and set data portion of new node
     QueueNode *newPtr = new QueueNode;
     newPtr->item = newItem;
     newPtr->next = NULL;
     if(isEmpty()) //insertion into empty queue
        frontPtr = newPtr;
     else          // insertion into nonempty queue
        backPtr->next = newPtr;
     backPtr = newPtr;
  }
  catch (bad_alloc e)
  {
     throw QueueExceptoin("QueueException: enqueue cannot allocate
                          memory.");
  }
} // end enqueue
```

```
void Queue::dequeue() throw (QueueException)
{
  if(isEmpty())
     throw QueueException("QueueException: empty queue, cannot
dequeue");
  else
  {  // queue is not empty; remove front
     QueueNode *tempPtr = fronPtr

     if(frontPtr == backPtr) // special case? A queue of one item.
     {  // yes, one node in queue
        frontPtr = NULL;
        backPtr = NULL;
     }
     else
        frontPtr = frontPtr->next;
     tempPtr->next = NULL;
     delete tempPtr;
  } // end if(isEmpty())
} // end dequeue
```

```
void Queue::dequeue(QueueItemType& queueFront) throw (QueueException)
{
  if(isEmpty())
     throw QueueException("QueueException: empty queue, cannot
dequeue");
  else
  {  // queue is not empty; retrieve front
     queueFront = frontPtr->item;
     dequeue();
  } // end if(isEmpty())
} // end dequeue
```

```
private:

   struct QueueNode
   {  QueueItemType  item;
      QueueNode      *next;
   } // end QueueNode

   QueueNode *backPtr;
   QueueNode *frontPtr;
}; // end Queue
```

```
#include "QueueP.h"

int main()
{
   Queue aQueue;
   aQueue.enqueue(15);
   ...
}
```

IMPLEMENTATION

- **An Array-based Implementation**
  - For applications in which a fixed-sized queue does not present a problem

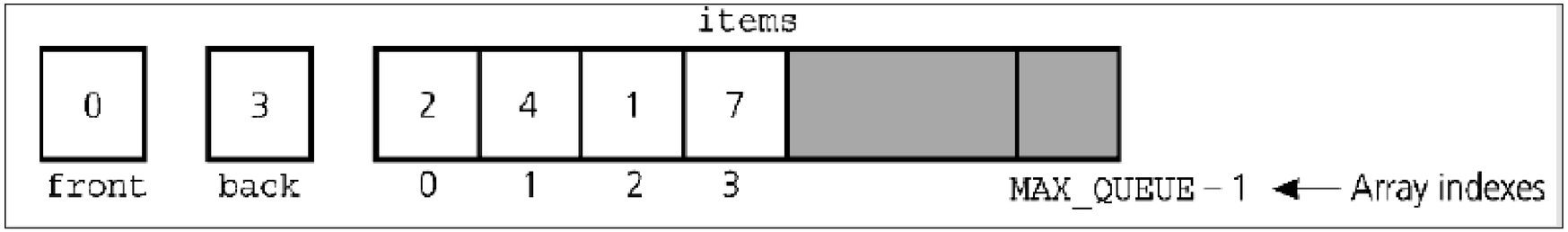
```
const int MAX_QUEUE = 100;
typedef int QueueItemType;

class Queue
{
public:
  ...
private:
    QueueItemType items[MAX_QUEUE];
    int front;
    int back;
}
```

The indexes of the front and back respectively, in the queue.

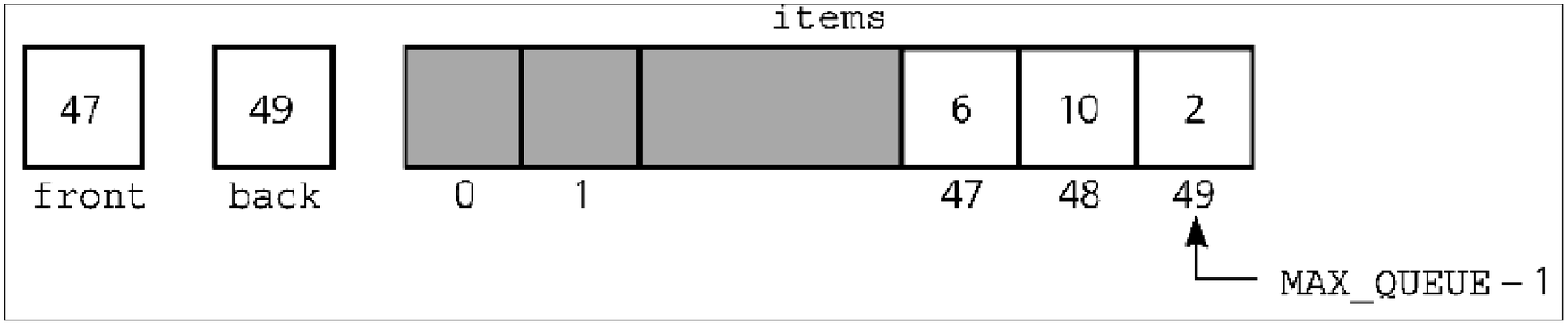
IMPLEMENTATION

- **An Array-based Implementation**
  - Initially, front is 0 and back is -1



  - To **insert** a new item, increment back and place the item in **items[back]**
  - To **delete** an item, simply increment front
  - The queue is **empty** whenever **back is less than front**

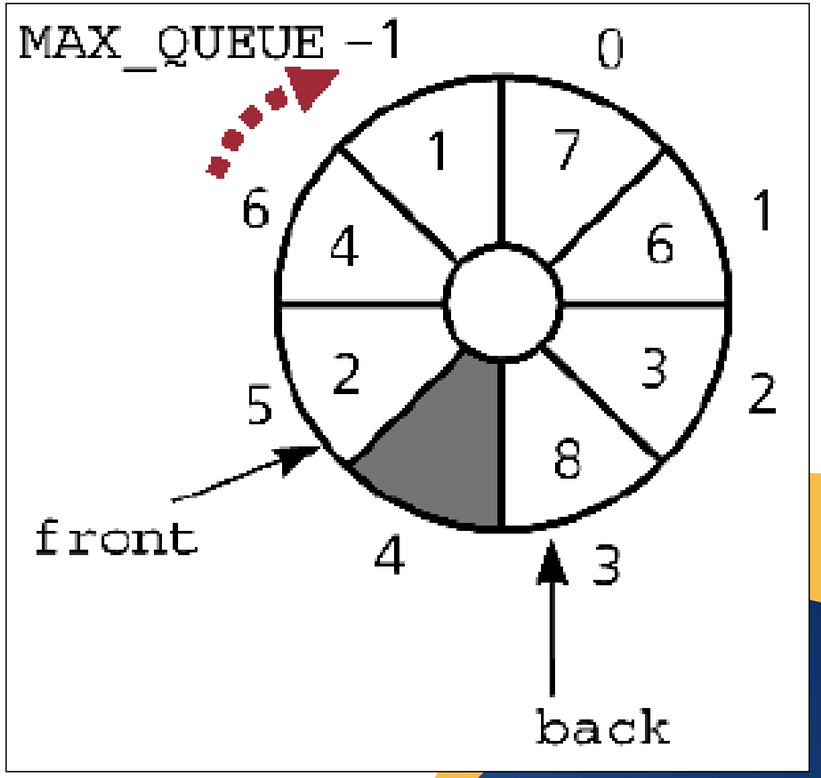
- The problem of **rightward drift**:
  - After a sequence of additions and removals, the items will drift towards the end of the array.



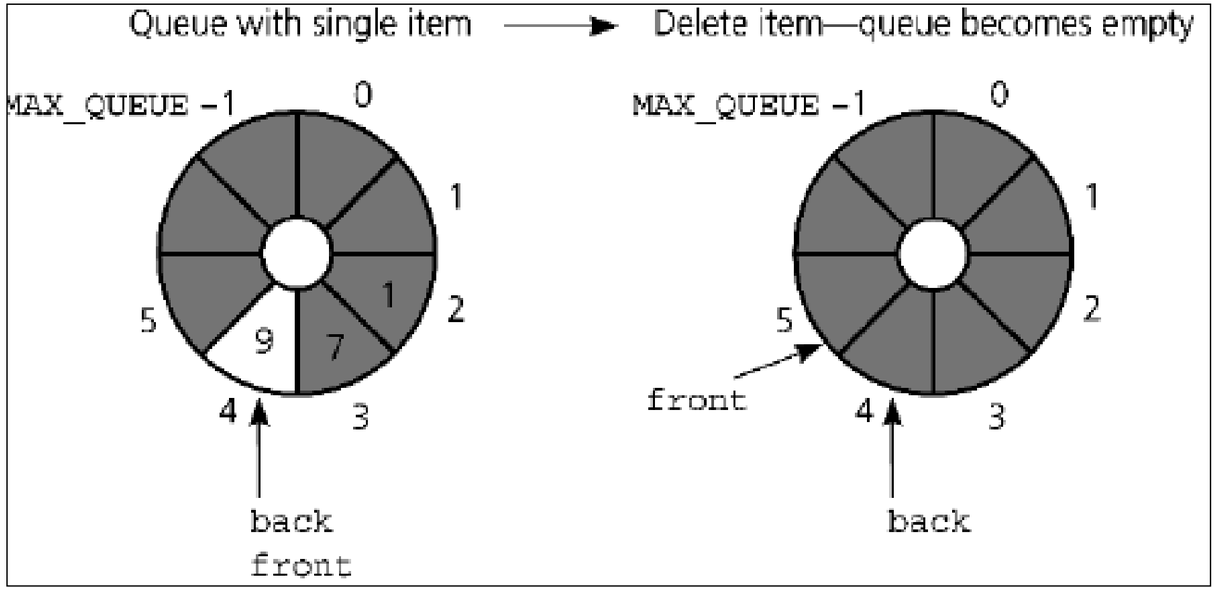
  - Back could equal MAX_QUEUE-1 even when the queue contains only a few items. Shifting elements to compensate for rightward drift is expensive.

IMPLEMENTATION

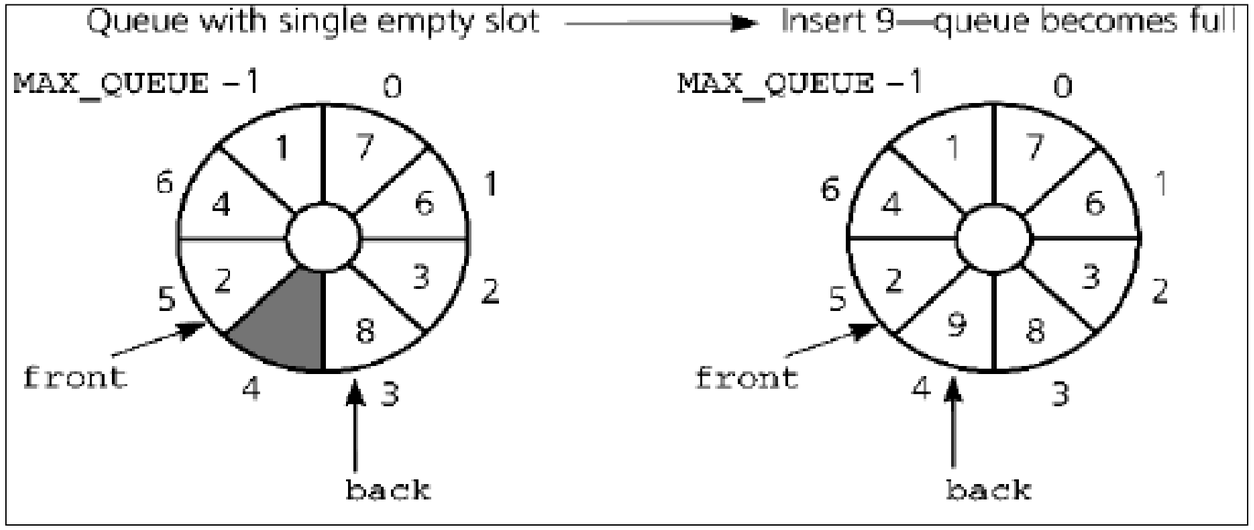
- **A circular array:**
  - Eliminates the problem by moving the index **clockwise** around the array.
  - But front and back cannot be used to distinguish between **queue-full** and **queue-empty** conditions



- **Queue-empty condition** when the front is one slot ahead of back.



- **Queue-full condition**:



IMPLEMENTATION

- **A circular array:**
  - To detect queue-full and queue-empty conditions, keep a count of the queue items
  - To initialize the queue, set:
    - `front` to `0`
    - `back` to `MAX_QUEUE - 1`
    - `count` to `0`

IMPLEMENTATION

- **An Array-based Implementation**

```
typedef int QueueItemType;
class Queue
{
public:
  Queue(); // default constructor
  bool isEmpty() const;
  void enqueue(const QueueItemType& newItem)
throw(QueueException);
  void dequeue() throw(QueueException);
  void dequeue(QueueItemType& queueFront) throw(QueueException);
  void getFront(QueueItemType& queueFront) const
        throw(QueueException);
```

```
private:
   QueueItemType items[MAX_QUEUE];
   int front;
   int back;
   int count;
}; // end Queue
```

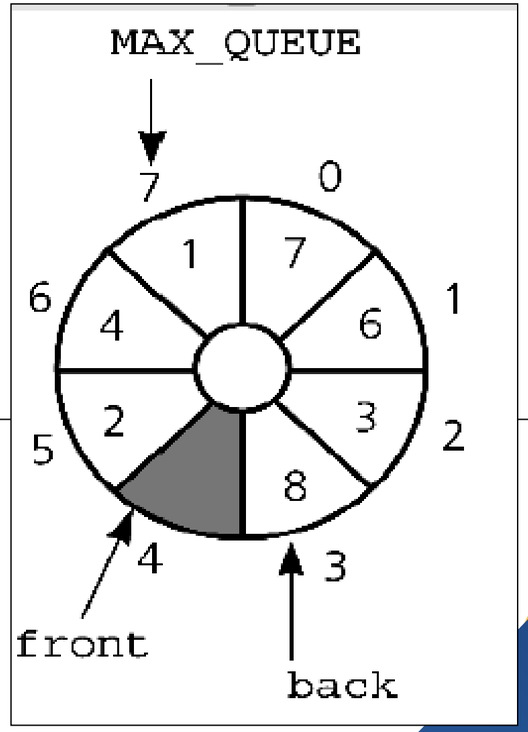
```
/** @file QueueA.cpp */
#include "QueueA.h"

Queue::Queue() : front(0), back(MAX_QUEUE-1), count(0)
{
} // end default constructor
```

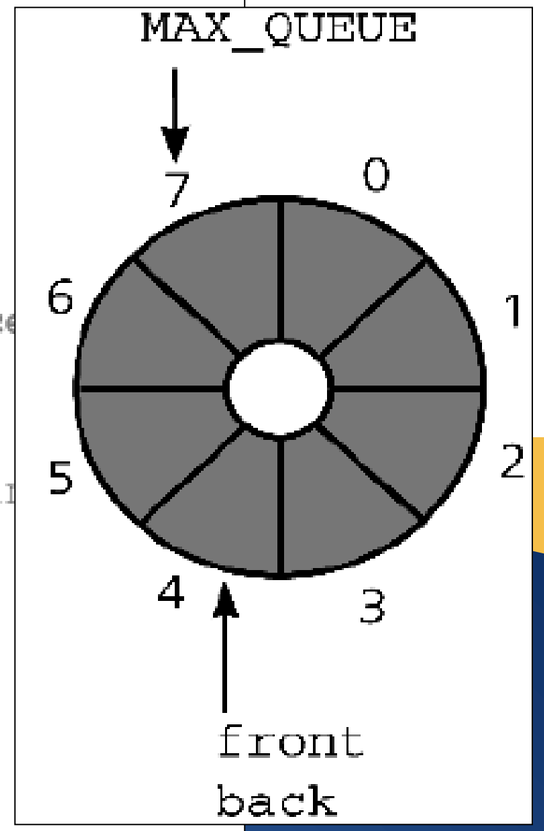
```
Queue::isEmpty() const
{
    return count == 0;
} // end isEmpty

void Queue::getFront(QueueItemType& queueFront) const
                    throw(QueueException)
{
    if(isEmpty())
        throw QueueException(
         "QueueException: empty queue, cannot getFront");
    else
        // queue is not empty; retrieve front
        queueFront = items[front];
} // end getFront
```

```
void Queue::enqueue(const QueueItemType& newItem)
        throw(QueueException)
{
    if(count == MAX_QUEUE)
        throw QueueException(
         "QueueException: queue full on enqueue");
    else
    {   back = (back+1) % MAX_QUEUE;
        items[back] = newItem;
        ++count;
    } // end if
} // end enqueue
```



```
void Queue::dequeue()  throw(QueueException)
{
    if(isEmpty())
         throw QueueException("QueueException: empty queue, cannot
dequeue");
    else
    {    front = (front+1) % MAX_QUEUE;
         --count;
    } // end if
} // end dequeue
void Queue::dequeue(QueueItemType& queueFront) throw(QueueExce
{
    if(isEmpty())
         throw QueueException("QueueException: empty queue, ca
dequeue");
    else
    {    queueFront = items[front];
         front = (front+1) % MAX_QUEUE;
         --count;
    } // end if
} // end dequeue
```



IMPLEMENTATION

- **ADT List Implementation**
  - The ADT queue uses an instance of List to represent the queue.
  - Using ADT List to implement queue is very simple and easy to write.

```
/** @file QueueL.h */
#include "ListP.h" // ADT list operations

class Queue
{
...
private:
   List aList; // list of queue items
};
```

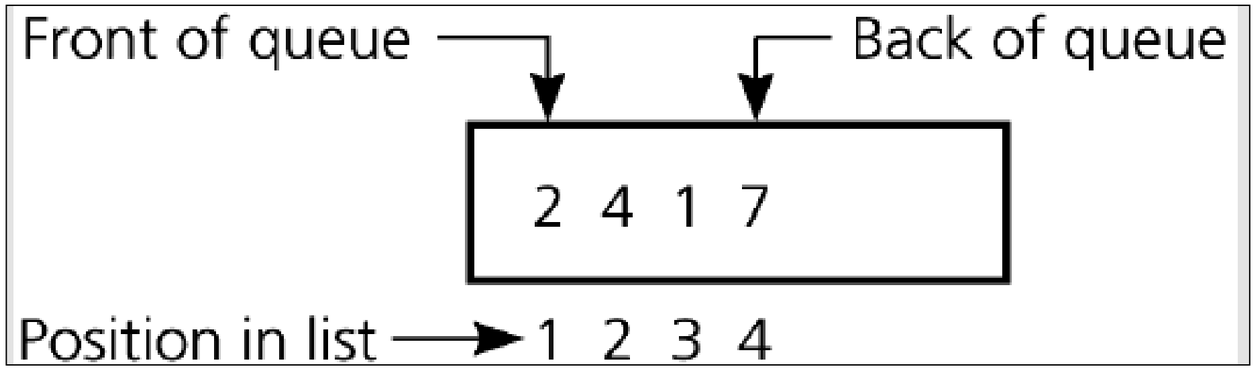
- The front of the queue is at position 1 of the list

  `deQueue()` as the list operation `remove(1)`

  `getFront (queueFront)` as the list operation `retrieved (1, queueFront)`

- The back of the queue is at the end od the list

  `enQueue(newItem)` as the list operation `insert(getLength()+1, newItem)`



- **Comparing Implementations**
  - **Fixed size vs. dynamic size**
    - A statically allocated **array-based implementation**
      - A fixed-size queue that can get full
    - A **pointer-based implementation**
      - No size restriction on the queue.
      - Use a pointer-based implementation is more efficient
      - ADT List approach is much simpler to write and saves programming time.

SUMMARY

- A **queue** is a linear data structure that follows the **First In, First Out (FIFO)** principle, meaning the first element added is the first to be removed.
- Common operations include:
  - `enqueue`: Adding an element to the back of the queue.
  - `dequeue`: Removing an element from the front.
  - `getFront`: Viewing the front element without removing it.
  - `isEmpty`: Checking if the queue is empty.

SUMMARY

- Queues are widely used in real-world applications like CPU scheduling, print queues, and handling tasks in service systems (e.g., customer support).
- Variants include **circular queues**, **priority queues**, and **double-ended queues (deque)**.

SUMMARY

- **A summary of Position-oriented ADT**
  - Position-oriented ADTs:
    - **List, Stack, Queue**
  - **Stack and Queue**: only the end positions can be accessed
  - **List**: all positions can be accessed.
  - We can insert into, delete from, and inspect the item at any position of the **ADT List.**

SUMMARY

- Stack and Queue are very similar:
  - Operations can be paired off as:
    - `createStack` and `createQueue`
    - Stack `isEmpty` and queue `isEmpty`
    - `push` and `enQueue`
    - `pop` and `deQueue`
    - Stack `getTop` and queue `getFront`